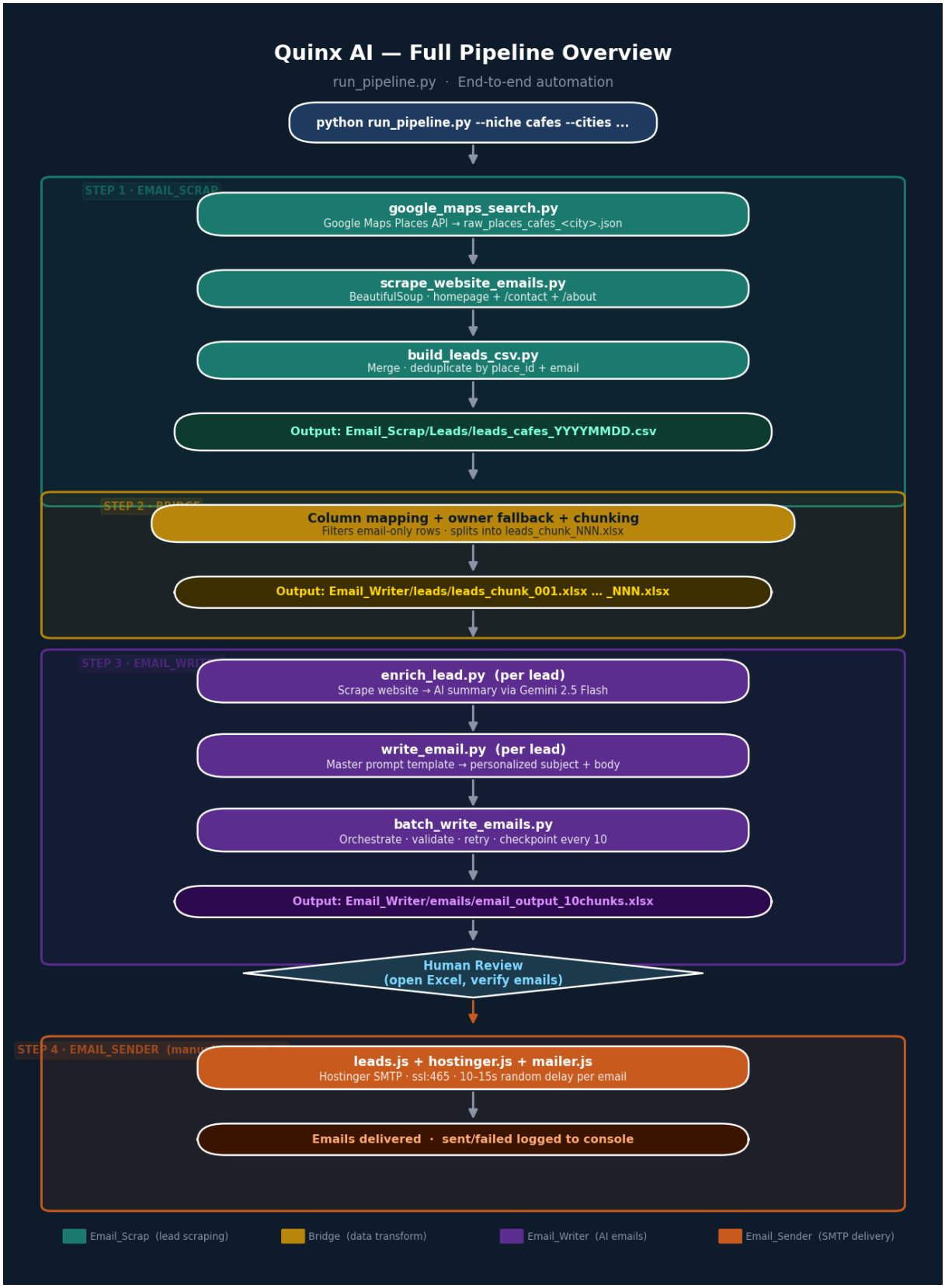
Quinx AI

Automation Architecture

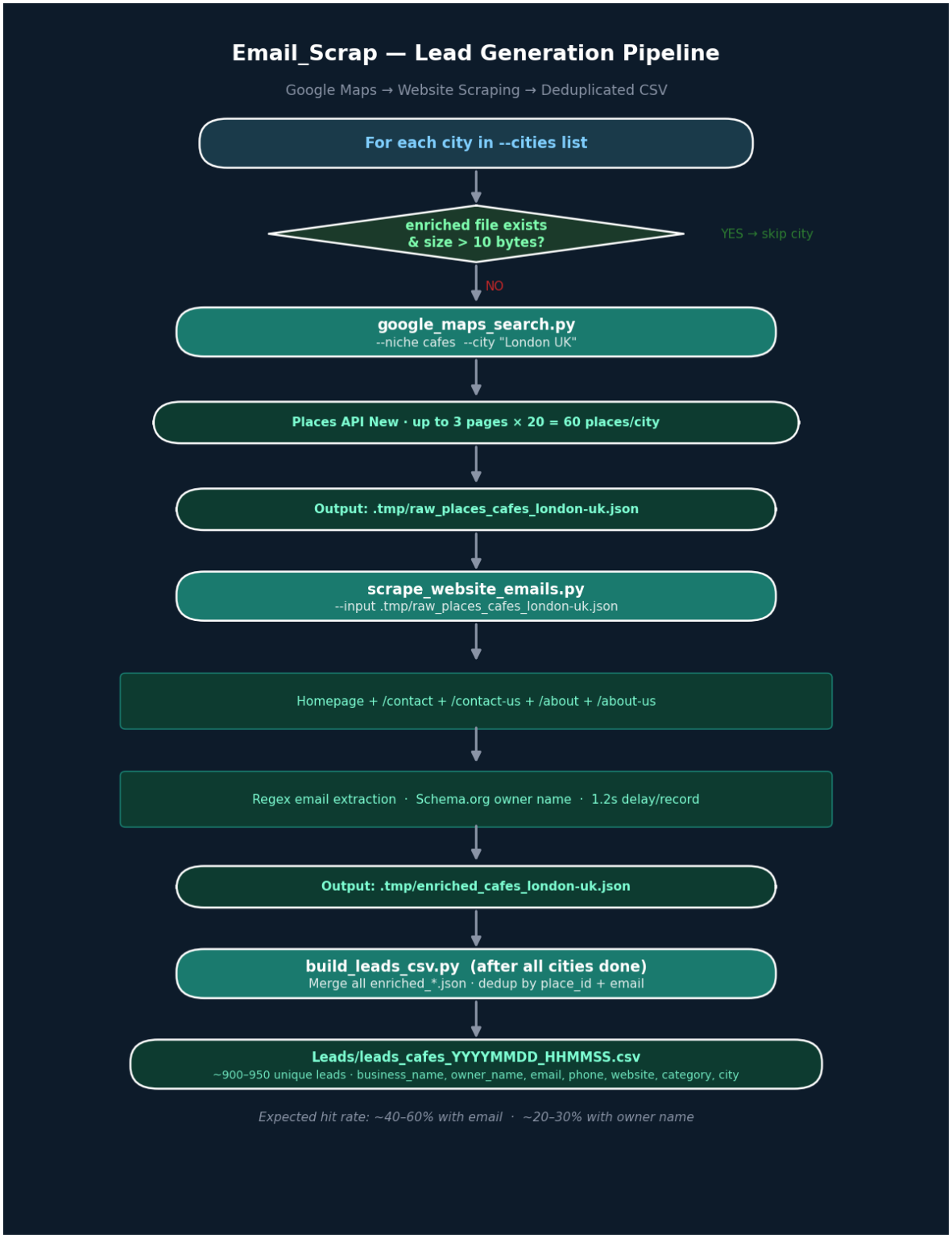
System	End-to-end cold outreach pipeline
Components	quinx-gui (FastAPI :8001 + React :5173) | Email_Scrap | Email_Writer | Email_Sender
Entry point	http://localhost:5173 (Quinx GUI -- browser-based control panel)
AI Model	OpenRouter (multi-model, key rotation) + Anthropic backup
SMTP	Hostinger · smtp.hostinger.com:465
Output	Personalized cold emails delivered to leads

Quinx GUI orchestrates the full pipeline: Campaign -> Scrape -> Write -> Send -> Logs.
Backend (FastAPI + SQLite) on :8001. Frontend (React + Vite + Tailwind) on :5173. Tasks: threads.

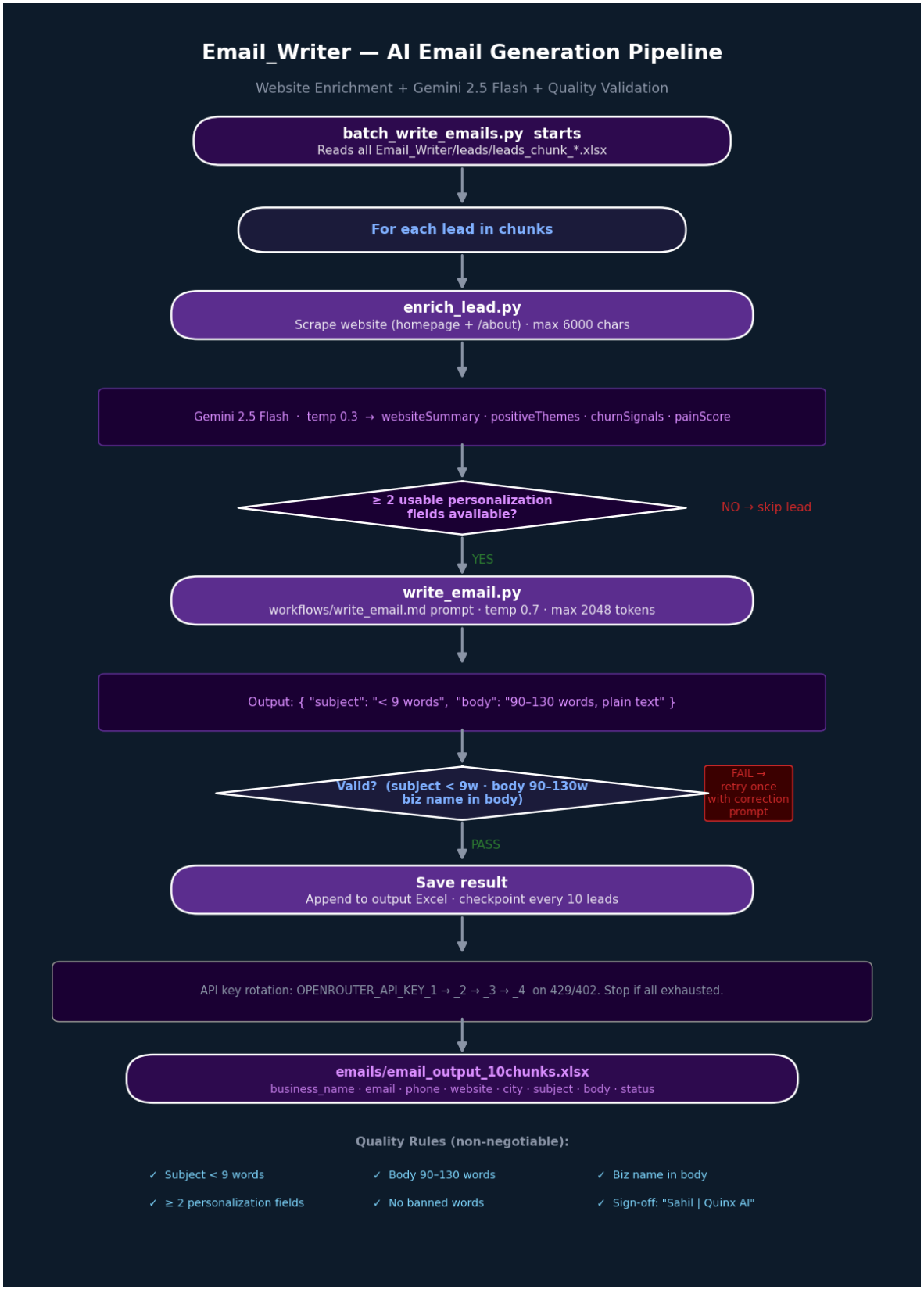
Full Pipeline Overview



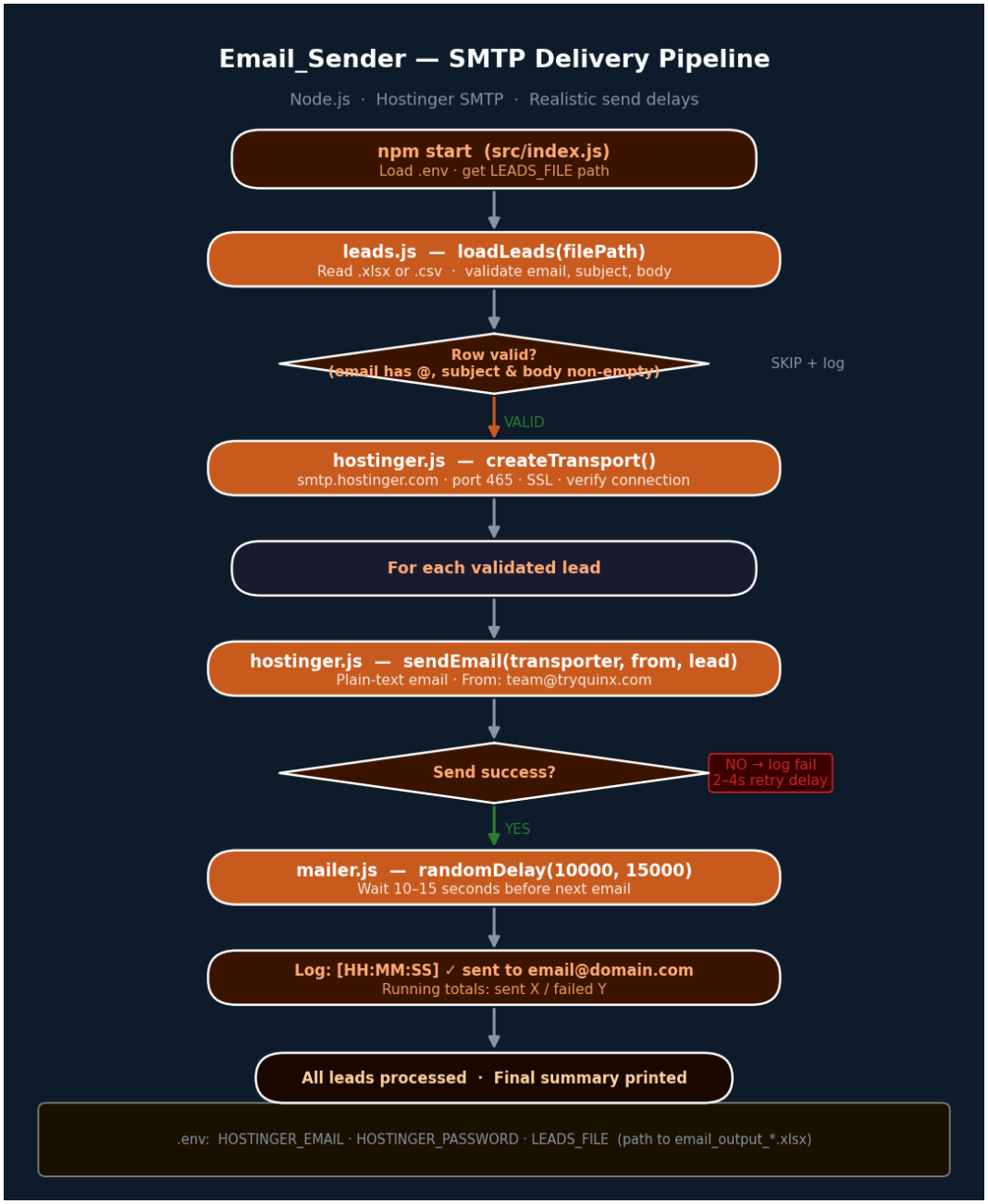
Step 2 -> Email_Scrap -> Lead Generation



Step 3 · Email_Writer — AI Email Generation



Step 4 · Email_Sender — SMTP Delivery



Data Dictionary

Stage	Fields
Raw (Maps API)	place_id · business_name · address · phone · website · types · city
Enriched (Scraper)	+ email · owner_name
Leads CSV (Email_Scrap)	business_name · owner_name · email · phone · website · category · city · source
Chunk XLSX (Bridge)	business_name · owner_name* · email · phone · website · city · category · niche * fallback: "<Name>'s Team" if blank
Writer Output (Email_Writer)	+ websiteSummary · positiveThemes · churnSignals · painScore · subject · body · status
Sender Input (Email_Sender)	email · subject · body (only these 3 are consumed)

API Keys Reference

Service	Environment Variable	Location	Used by
Google Maps Places API	GOOGLE_MAPS_API_KEY	Email_Scrap/.env	google_maps_search.py
OpenRouter (multi-model, key rotation)	OPENROUTER_API_KEY_1	Email_Writer/.env	enrich_lead.py write_email.py
Hostinger SMTP	HOSTINGER_EMAIL HOSTINGER_PASSWORD	quinx-gui DB	sender.py -> node

Quick Start

Start backend

```
cd quinx-gui/backend && venv/Scripts/uvicorn main:app --port 8001 --reload
```

Start frontend

```
cd quinx-gui/frontend && npm run dev
```

Open browser

```
http://localhost:5173
```

GUI workflow (Campaign -> Scraper -> Writer -> Sender -> Logs)

```
Each step streams live logs to the UI and has a Stop button
```